PulseTrack

Employee Productivity Tracker

Complete Technical Documentation — v1.0.0

Property	Value
Version	1.0.0
Angular	21.2.x (standalone)
Firebase SDK	v11 (direct, no @angular/fire)
Electron	33.x
Node.js required	22+
TypeScript	5.8+
Screenshot storage	Supabase Storage (free, no card)
Activity data	Cloud Firestore
Hosting	Vercel (Angular app)

1. Project Overview

PulseTrack is a full-stack employee productivity monitoring application similar to TimeDoctor. It consists of two separate codebases:

1. **Angular web app** — the admin dashboard where managers view charts, screenshots, reports, and manage employees. Deployed on Vercel.
2. **Electron desktop agent** — a silent background app installed on employee machines. Captures screenshots and tracks active app usage, uploading data to Firebase and Supabase.

Design goals

Zero cost — runs entirely on free tiers (Firebase Spark, Supabase free, Vercel hobby)
No custom backend — Firebase Auth + Firestore + Supabase Storage replace any server
Modern Angular — signals, OnPush, standalone components, functional guards throughout
Privacy-aware — employees log in with their own credentials; Firestore rules scope data by userId

1.1 Tech stack

Layer	Technology	Purpose
Frontend	Angular 21.2 standalone	Web dashboard UI
State	Angular Signals	Reactive state — no NgRx needed
Styling	SCSS + CSS variables	Dark theme design system
Auth	Firebase Auth v11	Email/password login for web + agent
Database	Cloud Firestore v11	Activity logs, user profiles, screenshot metadata
File storage	Supabase Storage	Screenshot image files (1 GB free, no card)
Hosting	Vercel	Auto-deploy from GitHub
Desktop agent	Electron 33 + Node.js 22	Screen capture + active app tracking
Charts	Chart.js	Productivity and usage visualisations
Image compression	sharp	JPEG compression in Electron agent
Session persistence	electron-store	Keeps agent logged in across reboots

2. System Architecture

2.1 High-level data flow

Three layers communicate exclusively through Firebase and Supabase. There is no custom backend server anywhere in the system.

Layer 1 — Employee machine (Electron agent)

Runs silently in the system tray after one-time login
Activity loop (every 5s): reads OS foreground window via active-win library
When active app changes: flushes session to Firestore activities collection with duration + category
Screenshot loop (every 30s): captures full screen, compresses to ~60KB JPEG via sharp, uploads to Supabase Storage, writes metadata to Firestore screenshots collection

Layer 2 — Firebase + Supabase (cloud, free tier)

Firebase Auth: handles login for both web dashboard and Electron agent
Firestore: stores activity logs, screenshot metadata, and user profiles with security rules
Supabase Storage: stores screenshot image files — 1 GB free, no credit card required
Firestore rules: employees can only write their own userId; admins can read everything

Layer 3 — Admin web app (Angular on Vercel)

Reads Firestore in real time via onSnapshot listeners — dashboard updates live
Dashboard: productivity score, hourly bar chart, doughnut chart, top apps table
Screenshots: paginated timeline of images (URLs from Supabase) grouped by hour
Reports: productivity table per employee, app usage chart, URL usage, CSV export
Employees: invite, list, detail page, deactivate — admin-only routes

2.2 Firestore collections

Collection	Document fields	Written by	Read by
users	uid, email, displayName, role, teamId, active, createdAt	Admin (invite form)	Own doc (any), All (admin)
activities	userId, appName, windowTitle, url?, category, startTime, endTime, durationSeconds	Electron agent	Own docs (employee), All (admin)
screenshots	userId, imageUrl, capturedAt, appName, windowTitle	Electron agent	Own docs

			(employee), All (admin)

2.3 Firestore security rules

Defined in Firebase Console > Firestore > Rules. Key patterns:

- users: read own doc only (or admin reads all); only admin can write
- activities: employees create only where userId == their own uid; admin updates/deletes
- screenshots: any authenticated user can create; only admin can delete
- isAdmin() helper: reads the requesting user Firestore profile and checks role == "admin"

Critical: Firestore composite indexes

The following composite indexes must exist — create them in Firebase Console > Firestore > Indexes:

activities: userId ASC + startTime DESC
activities: startTime DESC (team query, no userId filter)
screenshots: userId ASC + capturedAt ASC
screenshots: capturedAt ASC (admin all-team query)

If missing, queries fail with "requires index" error. Click the URL in the error to auto-create.

3. Angular Web App

3.1 Folder structure

```
src/app/
├── core/
│   ├── firebase.ts              ← Firebase singleton (app, auth, db)
│   ├── error-handler.ts         ← Global Angular ErrorHandler
│   └── services/
│       ├── auth.service.ts      ← Auth signals + login/logout
│       ├── activity.service.ts  ← Firestore queries + onSnapshot listeners
│       ├── screenshot.service.ts← Screenshot Firestore queries
│       ├── employee.service.ts  ← User CRUD
│       └── toast.service.ts     ← Global toast signal
├── features/
│   ├── auth/                    ← Login page + auth.routes.ts
│   ├── dashboard/               ← Overview charts + dashboard.routes.ts
│   ├── employees/               ← List, detail, invite + employees.routes.ts
│   ├── screenshots/             ← Timeline viewer + screenshots.routes.ts
│   └── reports/                 ← Productivity, app, URL + reports.routes.ts
├── guards/
│   ├── auth.guard.ts            ← Waits for authReady, redirects if not logged in
│   └── admin.guard.ts           ← Redirects non-admins to dashboard
└── shared/components/
    ├── shell/                   ← Sidebar + topbar authenticated layout
    ├── date-range/              ← Reusable date range selector (Today/7d/30d/Custom)
    └── toast/                   ← Toast notification renderer
```

3.2 Angular 21 rules — applied everywhere

Rule	Implementation
Standalone components	No NgModules. Every component declares its own imports array.
Signals	signal(), computed() for all state. No BehaviorSubject or Observable state.
inject()	All DI via inject(). No constructor parameters.
OnPush CD	ChangeDetectionStrategy.OnPush on every component without exception.
effect() + untracked()	Side effects triggered by signal changes. Signal writes inside effect() always wrapped in untracked().
Functional guards	authGuard and adminGuard are plain functions — not classes.
Native control flow	@if, @for, @switch in all templates. Never *ngIf or *ngFor.
input() / output()	Component I/O functions. Never @Input() / @Output() decorators.
toSignal()	Converts Router event Observables to signals in ShellComponent.

afterNextRender()	Chart.js canvas init. Prevents SSR issues.

3.3 AuthService

src/app/core/services/auth.service.ts — the central auth hub. Uses Firebase onAuthStateChanged.

Member	Type	Description
firebaseUser	signal<User|null>	Raw Firebase Auth user
appUser	signal<AppUser|null>	Full profile from Firestore users collection
isLoading	signal<boolean>	True while auth resolves on startup
authReady	signal<boolean>	True once onAuthStateChanged fires for the first time
isLoggedIn	computed<boolean>	Derived from firebaseUser
isAdmin	computed<boolean>	True if appUser.role === "admin"
login(email, password)	Promise<void>	signInWithEmailAndPassword then navigate to /dashboard
logout()	Promise<void>	signOut then navigate to /auth/login

Why authReady matters

Without authReady, components mount and query Firestore before onAuthStateChanged resolves.
Result: empty userId, empty query results, "No data" shown even when data exists.
Fix: every data-loading effect checks if (!auth.authReady()) return; before proceeding.
The authGuard also waits for authReady via toObservable().pipe(filter(ready => ready), take(1)).

3.4 ActivityService

src/app/core/services/activity.service.ts — all Firestore queries for activity data.

Method	Returns	Used by
getActivityForUser(uid, from, to)	Promise<ActivityLog[]>	Reports (one-shot)
getTeamActivitySummary(from, to)	Promise<ActivityLog[]>	Reports admin (one-shot)
listenActivityForUser(uid, from, to, cb)	Unsubscribe fn	Dashboard (real-time)
listenTeamActivity(from, to, cb)	Unsubscribe fn	Dashboard admin (real-time)
getDailyProductivityScore(logs)	number 0-100	Stat card, reports table

groupByApp(logs)	Array<{appName, totalSeconds}>	Top apps table, bar chart
groupByHour(logs, date)	Array<{hour, productiveSeconds, totalSeconds}>	Hourly bar chart
groupByDomain(logs)	Array<{domain, totalSeconds, visitCount}>	URL usage report
seedFakeActivities(userId)	Promise<void>	Dev/testing only via browser console

3.5 Dashboard — real-time updates

- Uses listenActivityForUser / listenTeamActivity (onSnapshot) not getDocs
- Stores the Unsubscribe function: private unsubscribe: (() => void) | null = null
- Calls unsubscribe?.() before each new listener and in ngOnDestroy
- Shows a pulsing green "Live" indicator badge when listener is active
- Two Chart.js charts via afterNextRender() — destroyed and recreated on data refresh
- Four stat cards: productivity score (color-coded green/amber/red), active time, top app, sessions

3.6 Reports module

Route	What it shows	Special features
/reports/productivity	Per-employee table: score, active time, top app, sessions	CSV export, date range filter
/reports/apps	Top apps bar chart + categorised table	Employee filter dropdown (admin only)
/reports/urls	Top domains from activity logs with url field	Grouped by domain, sorted by time

All three share DateRangeComponent (Today / 7 days / 30 days / Custom date picker). CSV export is client-side only using Blob + URL.createObjectURL.

3.7 Shell layout

- ShellComponent wraps all authenticated feature routes as a parent route
- 240px sidebar: brand dot + "PulseTrack", nav items, user avatar (first letter), logout button
- Nav items: Dashboard, Screenshots, Reports always visible; Employees only if isAdmin()
- 60px topbar: dynamic page title from Router events via toSignal(), Live badge, user email

- Routing: each feature routes.ts wraps ShellComponent as parent with loadComponent children

4. Electron Desktop Agent

4.1 Folder structure

```
pulsetrack-agent/
├── src/
│   ├── main.ts          ← Electron main process, tray icon, orchestration
│   ├── firebase.ts      ← Firebase singleton (same project as Angular app)
│   ├── supabase.ts      ← Supabase client (reads SUPABASE_URL + ANON_KEY from config)
│   ├── auth.ts          ← Login BrowserWindow + Firebase Auth IPC handlers
│   ├── tracker.ts       ← Active app tracking loop (every 5s)
│   ├── screenshot.ts    ← Screenshot capture + upload loop (every 30s)
│   ├── uploader.ts      ← uploadActivity() to Firestore, uploadScreenshot() to
Supabase
│   ├── config.ts        ← dotenv loader, Firebase config, app categorisation arrays
│   └── store.ts         ← electron-store: session persistence across restarts
├── assets/tray-icon.png ← 32x32 tray icon
├── scripts/create-icon.ts ← One-time script to generate tray icon via sharp
├── .env                 ← Credentials — NEVER commit to git
├── .gitignore           ← Excludes .env and dist/ and release/
├── DEPLOY.md            ← IT deployment guide for distributing to employees
├── package.json
└── tsconfig.json
```

4.2 Two independent tracking loops

Loop 1 — Activity tracker (src/tracker.ts) — runs every 5 seconds

Calls active-win to get the OS foreground window: appName + windowTitle
If same app as currentSession: does nothing (session still running)
If app changed: calculates duration = now - session.startTime
If duration >= 5s: calls uploadActivity() to write to Firestore activities collection
Starts new session with the new app and current timestamp
categoriseApp() in config.ts classifies the app as productive / unproductive / neutral
This loop drives ALL analytics — productivity score, top apps, charts, reports

Loop 2 — Screenshot capture (src/screenshot.ts) — runs every 30 seconds

Uses screenshot-desktop to capture the full screen as a PNG Buffer
Passes through sharp: resize to max 1280px wide, JPEG at 60% quality (~60KB result)
Also reads the active window at capture time for metadata
Calls uploadScreenshot(): uploads JPEG to Supabase screenshots/{userId}/{timestamp}.jpg
Gets the Supabase public URL and writes a Firestore doc to screenshots collection
These loops are completely independent — disabling screenshots does not affect analytics

4.3 Login and startup flow

1. app.whenReady(): create tray icon, call setupLoginIpc(), call getCurrentUser()
2. getCurrentUser(): wraps onAuthStateChanged in a Promise — resolves with user or null
3. If user exists (session restored): call onUserLoggedIn() immediately
4. If no user: call openLoginWindow() — shows 400x500 BrowserWindow with dark HTML login form
5. Employee submits form: ipcRenderer.send("login-attempt", { email, password })
6. Main process: ipcMain.on("login-attempt") calls signInWithEmailAndPassword
7. On success: close window, call onUserLoggedIn(user)
8. onUserLoggedIn(): saveSession() to electron-store, enableAutoLaunch(), startTracking(user), startScreenshots(user), update tray menu

4.4 .env file — all configuration

pulsetrack-agent/.env — bundled into the installer as an extraResource. Never committed to git.

```
FIREBASE_API_KEY=your_firebase_api_key
FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_MESSAGING_SENDER_ID=your_sender_id
FIREBASE_APP_ID=your_app_id
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your_supabase_anon_key
SCREENSHOT_INTERVAL_MS=30000
ACTIVITY_INTERVAL_MS=5000
```

Security note on bundled credentials

Firebase API key and Supabase anon key are safe to bundle — they identify the project, not grant access.
Real security: Firestore rules enforce userId scoping. No valid Firebase Auth session = no writes.
Supabase Storage policies restrict uploads to authenticated requests with bucket_id = "screenshots".
If keys are ever compromised: rotate them in Firebase Console / Supabase, rebuild and redistribute.

4.5 App categorisation

In src/config.ts, two string arrays determine the productivity category of each app:

- PRODUCTIVE_APPS: code, cursor, webstorm, intellij, figma, notion, terminal, outlook, slack, zoom, teams, word, excel, etc.
- UNPRODUCTIVE_APPS: youtube, netflix, spotify, twitter, reddit, instagram, tiktok, discord, twitch, etc.
- categoriseApp(appName): toLowerCase() + Array.some(a => lower.includes(a)) check
- No match in either array = neutral

To reclassify apps: edit config.ts arrays, bump package.json version, rebuild and redistribute.

4.6 Build and distribution commands

Command	Output file	Platform
`npm run package:win`	release/Setup 1.0.0.exe	Windows x64 (NSIS)
`npm run package:mac`	release/PulseTrack Agent-1.0.0.dmg	macOS x64 + arm64
`npm run package:linux`	release/PulseTrack Agent-1.0.0.AppImage	Linux x64
`npm run package:all`	All three above	All platforms

- Windows installer: oneClick: true, perMachine: false — silent install, no admin rights needed
- macOS: requires Screen Recording + Accessibility permissions on first run
- .env bundled as extraResources — read from process.resourcesPath when app.isPackaged

5. Build History — All 8 Phases

Phase	What was built	Status
1 — Firebase + scaffold	Firebase project setup, Firestore/Storage rules, Angular 21 project, routing with lazy loading, functional guards (authGuard, adminGuard), AuthService with signals, environment config	Complete
2 — Login + shell	Login page (dark theme, grid background, animated card, loading spinner signal), ShellComponent (sidebar 240px, topbar 60px, brand, nav with SVG icons, user avatar, logout)	Complete
3 — Dashboard	ActivityService (queries + helpers), Chart.js doughnut + bar charts via afterNextRender, 4 stat cards, date range pills, authReady timing fix, Firestore composite index creation	Complete
4 — Screenshots	Screenshots timeline page, paginated grid of images, lightbox viewer, grouping by hour, screenshot metadata from Firestore + images from Supabase URLs	Complete
5 — Employees	Employee list page (search, filter, status badges), invite form, employee detail page (stats + recent screenshots), deactivate action, admin-only routing via adminGuard	Complete
6 — Reports	ReportsLayoutComponent with tab nav, ProductivityComponent (per-employee table + CSV export), AppUsageComponent (bar chart + employee filter), UrlUsageComponent (domain grouping), DateRangeComponent	Complete
7 — Electron agent	Activity tracking loop (active-win, 5s), screenshot loop (screenshot-desktop + sharp, 30s), Firebase Auth login window (IPC), Supabase Storage upload, system tray menu, auto-launch, electron-store session persistence	Complete
8 — Production polish	Real-time onSnapshot listeners on dashboard, live indicator badge, GlobalErrorHandler, ToastService + ToastComponent, dynamic page titles via toSignal, loading screen in index.html, vercel.json with SPA rewrite, firestore.indexes.json	Complete

6. Phase 9 — Planned Features

Three additions planned for the next release (v1.1.0). All are backwards-compatible with existing data.

6.1 Auto-update via electron-updater

Enables silent self-updating. Agents on employee machines check for new versions and install them automatically.

New dependency

```
npm install electron-updater
```

New file: src/updater.ts

- Import autoUpdater from electron-updater
- Set autoUpdater.autoDownload = true and autoUpdater.autoInstallOnAppQuit = true
- Call autoUpdater.checkForUpdates() on startup and every 4 hours
- Log update events: checking-for-update, update-available, update-downloaded

Changes to src/main.ts

- Import and call initUpdater() after app.whenReady() resolves

Changes to package.json build section

```
"publish": [{ "provider": "github", "owner": "YOUR_GITHUB_USERNAME", "repo":
"pulsetrack-agent" }]
```

Release workflow for every future update

9. Make code changes in src/
10. Bump version in package.json (e.g. 1.0.0 -> 1.1.0)
11. Run: npm run package:win (or mac/linux)
12. Create a GitHub Release tagged v1.1.0
13. Upload installer + latest.yml files to the release
14. Agents on employee machines auto-download and install within 4 hours

6.2 URL tracking — active browser tab

Tracks which specific URLs employees visit, not just "Chrome is open". Uses a Chrome extension + Native Messaging to communicate with the Electron agent.

How it works

- Chrome extension: background service worker listens to chrome.tabs.onActivated and chrome.tabs.onUpdated

- When tab changes: sends { url, title } to the Electron agent via chrome.runtime.sendNativeMessage
- Electron agent: registered as a Native Messaging host, receives URL messages and attaches the url field to the current activity session before writing to Firestore
- Angular app: URL usage report at /reports/urls already reads the url field — zero Angular changes needed

New files to create

- pulsetrack-agent/src/native-messaging.ts — registers native host, listens for URL messages
- chrome-extension/manifest.json — Chrome extension manifest v3 with nativeMessaging permission
- chrome-extension/background.js — service worker: tab listener + sendNativeMessage
- chrome-extension/install.sh + install.bat — register native host path in OS registry/plist

Deployment note for URL tracking

The Chrome extension must be installed on each employee machine.

IT teams can force-deploy it via Google Workspace Admin > Chrome Browser Management.

The native messaging host manifest must be registered — the install script handles this automatically.

6.3 Configurable screenshot interval per employee

Admins can set different screenshot frequencies for different employees directly from the web dashboard. No rebuild required.

How it works end-to-end

- Add screenshotIntervalMs: number field to AppUser interface and to the users Firestore collection (default: 30000)
- Angular employee detail page: admin sees an interval selector dropdown (10s / 30s / 1min / 5min / Disabled)
- Saving the dropdown calls EmployeeService.updateScreenshotInterval(uid, intervalMs)
- Electron agent: after login, reads screenshotIntervalMs from users/{uid} via onSnapshot
- When the value changes: calls stopScreenshots() then startScreenshots(user, newIntervalMs)
- If intervalMs is 0: skips the screenshot loop entirely for that employee

Angular changes needed

- auth.service.ts: add screenshotIntervalMs?: number to AppUser interface
- employee.service.ts: add updateScreenshotInterval(uid, ms) method
- employees/detail component: add interval selector UI (admin only, hidden from employee view)

Electron agent changes needed

- config.ts: export DEFAULT_SCREENSHOT_INTERVAL_MS = 30000

- main.ts: after login, call startListeningToIntervalChanges(user)
- screenshot.ts: accept intervalMs param in startScreenshots(user, intervalMs)
- tracker.ts: new function listenToIntervalChanges(uid) using onSnapshot on users/{uid}

7. Developer Setup Guide

7.1 Prerequisites

Tool	Required version	Install
Node.js	22.x LTS minimum	nodejs.org/en/download
npm	10+ (bundled with Node)	—
Angular CLI	21.2.x	npm install -g @angular/cli
Git	Any recent version	git-scm.com

7.2 Angular web app — local dev

15. cd into the Angular project folder
16. Run: npm install
17. Fill in Firebase config in src/environments/environment.ts
18. Run: ng serve
19. Open: http://localhost:4200

Disable any ad blocker for localhost — it blocks Firestore WebSocket connections.

Production build: ng build --configuration production

7.3 Electron agent — local dev

20. cd into pulsetrack-agent/
21. Copy .env.example to .env (create it if missing — see Section 4.4 for all fields)
22. Run: npm install
23. Run: npm run dev
24. Login window appears — enter Firebase email/password
25. After 30s: check Supabase Storage for screenshot files
26. Check Firestore activities collection for new documents

macOS permissions required

Screen Recording: System Settings > Privacy & Security > Screen Recording > enable PulseTrack Agent

Accessibility: System Settings > Privacy & Security > Accessibility > enable PulseTrack Agent

Without these, screenshot-desktop and active-win will silently fail or return empty data.

7.4 Seeding test data (no agent needed)

27. Open the Angular app in the browser
28. Open DevTools > Console (F12)
29. Get your Firebase UID from Firebase Console > Authentication
30. Run in console: seedFakeActivities("YOUR_UID_HERE")
31. Wait for: "Seeded 50 fake activity logs" confirmation
32. Refresh the dashboard — charts and stats should populate

This creates 50 fake ActivityLog documents spanning the last 8 hours using a mix of all app categories.

8. Deployment

8.1 Angular app — Vercel

33. Push Angular project to GitHub
34. Go to vercel.com > New Project > Import repo
35. Framework preset: Other (not Angular — we manage the build ourselves)
36. Build command: npm run build
37. Output directory: dist/employee-tracker/browser
38. Click Deploy

The vercel.json at project root handles everything: build config + SPA rewrite rule (all routes -> index.html).

8.2 Electron agent — distributing to employees

39. Ensure .env has production Firebase + Supabase credentials
40. Run: npm run package:win (builds release/Setup 1.0.0.exe)
41. Share the .exe with employees via email, Google Drive, or IT deployment tool
42. Employee runs installer > accepts UAC prompt > PulseTrack Agent installs silently
43. Login window appears on first launch > employee enters their credentials > done
44. Agent starts automatically on every subsequent reboot

Windows installer details

oneClick: true — installs without showing a wizard, just a progress bar
perMachine: false — installs to AppData, no admin rights required
.env bundled as extraResources — read from process.resourcesPath at runtime
NSIS uninstaller created automatically — appears in Add/Remove Programs

8.3 Free tier limits

Service	Free limit	Estimated usage (10 employees)
Firebase Auth	Unlimited users	10 users — no concern
Firestore reads	50,000 / day	~5,000 / day — well within limit
Firestore writes	20,000 / day	~12,000 / day (activity + screenshot docs)
Firestore storage	1 GB	~50 MB / month for metadata
Supabase Storage	1 GB total files	~570 MB / day if all screenshots viewed (monitor!)
Supabase bandwidth	5 GB / month	Depends on how often screenshots page is opened

Vercel bandwidth	100 GB / month	Angular app bundle — no concern

Supabase bandwidth is the most likely constraint at scale. If exceeded, consider increasing the screenshot interval or implementing lazy-loading on the screenshots timeline page.

9. Troubleshooting

Problem	Cause	Fix
"No activity data yet" on dashboard	authReady race condition or missing Firestore index	Check effect() uses if (!auth.authReady()) return. Create missing indexes (click console URL).
"The query requires an index"	Missing Firestore composite index	Click the URL in the browser console error. Wait 2-3 min for index to build.
ERR_BLOCKED_BY_CLIENT on Firestore	Ad blocker intercepting Firebase WebSocket	Disable ad blocker for localhost and the production domain.
Screenshots not uploading	Supabase Storage policy missing or macOS permission denied	Add INSERT + SELECT policies on screenshots bucket. Grant Screen Recording permission on macOS.
Login window reappears every restart	electron-store not saving or Firebase token expired	Check store.ts saveSession() is called in onUserLoggedIn(). Check Firebase Auth token TTL.
Charts blank after data loads	Chart.js canvas reuse error	Ensure chartInstance.destroy() is called before creating a new chart on the same canvas.
Vercel: 404 on direct URL / page refresh	Missing Angular SPA rewrite rule	Confirm vercel.json has: { "source": "/(.*)", "destination": "/index.html" }
Agent not auto-launching on reboot	AutoLaunch not enabled or OS permission denied	Check enableAutoLaunch() is called. On macOS: check Login Items in System Settings.
allowSignalWrites deprecation warning	Old effect() option in Angular 21	Remove { allowSignalWrites: true }. Wrap signal writes in untracked(() => ...) instead.

10. Glossary

Term	Definition
ActivityLog	A Firestore document in the activities collection. Records one continuous session in one app. Key fields: userId, appName, windowTitle, url?, category, startTime, endTime, durationSeconds.
AppUser	A Firestore document in the users collection. Fields: uid, email, displayName, role (admin|employee), teamId, active, createdAt.
authReady	Angular signal on AuthService. Becomes true once onAuthStateChanged fires for the first time. Gates all Firestore queries in components.
category	Classification of an app session: productive, unproductive, or neutral. Set by categoriseApp() in config.ts via substring matching against two arrays.
electron-store	npm package that persists data to a JSON file on disk. Used to store the logged-in user session so the agent stays logged in across reboots.
onSnapshot	Firebase Firestore real-time listener. Returns an unsubscribe function. Fires immediately with current data and again on any change. Used on dashboard for live updates.
Productivity score	Percentage of total tracked seconds classified as productive. Computed by ActivityService.getDailyProductivityScore(logs). Color: green >70, amber 40-70, red <40.
Shell	ShellComponent — the Angular component providing the sidebar + topbar layout wrapper for all authenticated pages. Each feature's routes.ts uses it as a parent route with router-outlet.
Standalone component	Angular 17+ default where each component declares its own imports array instead of belonging to an NgModule. The only pattern used in this project.
Unsubscribe	Function returned by onSnapshot(). Must be called in ngOnDestroy to stop the Firestore listener and prevent memory leaks when navigating away.